

WEB APPLICATION ATTACK SIMULATION AND DEFENSE TOOLKIT (WEBXPLOIT)

***PROJECT REPORT Submitted in partial fulfillment of the requirements
for completion of the Cybersecurity Training Program***

Submitted By: Aditya Pramod Nikam

Course: Cybersecurity

Institute: FortuneCloud Private Ltd

Date: 21/09/2026

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope
6. Technologies Used
7. System Architecture
8. Laboratory Environment
9. Project Implementation
10. Attack Scenarios
11. Detection and Defense
12. Security Monitoring
13. Incident Response Workflow
14. Results
15. Challenges and Troubleshooting
16. Limitations
17. Future Enhancements
18. Conclusion
19. References

Abstract:-

WebXploit — Web Application Attack Simulation & Defense Toolkit

WebXploit is a controlled cybersecurity laboratory designed to demonstrate the complete lifecycle of web application attack simulation, detection, blocking, security telemetry collection, monitoring, and investigation. The project integrates offensive security testing with defensive security mechanisms in an isolated virtualized and containerized environment.

The laboratory uses **Kali Linux** as the security testing platform and **Ubuntu Server** as the primary server environment. **OWASP Juice Shop** is deployed as the intentionally vulnerable web application, while **Nginx with ModSecurity and the OWASP Core Rule Set (CRS)** provides the Web Application Firewall (WAF) layer. Controlled security scenarios including **Cross-Site Scripting (XSS), SQL Injection, network reconnaissance, Command Injection, and Path Traversal** are performed against the protected application.

When malicious requests are generated, ModSecurity and OWASP CRS inspect the HTTP traffic, identify patterns associated with attacks, calculate anomaly scores, and block requests that exceed the configured threshold. The WAF generates security telemetry containing information such as the request, source address, HTTP response, transaction information, and triggered security rules. This telemetry is intended to be collected using **Filebeat**, stored and analyzed using **Elasticsearch**, and visualized through **Kibana**.

The project follows an attack-and-defense workflow:

Attack Simulation → Detection → Blocking → Logging → Telemetry Collection → Security Monitoring → Investigation → Incident Response

The implementation demonstrates how web application security controls can be combined with centralized security monitoring to provide practical exposure to **Web Application Security, Web Application Firewalls, Detection Engineering, Security Operations, and Incident Response**. The project is designed exclusively for controlled laboratory use and provides a reproducible environment for cybersecurity learning and practical security analysis.

2.Introduction :-

Web applications have become an essential part of modern information systems, providing services such as online transactions, authentication, data processing, communication, and business operations. Because these applications are exposed through network and web interfaces, they can become targets for different types of security attacks. Common web application threats include **Cross-Site Scripting (XSS), SQL Injection, Command Injection, Path Traversal, and automated reconnaissance**.

Protecting web applications requires more than identifying vulnerabilities. A practical security environment should also demonstrate how malicious requests are **detected, analyzed, blocked, logged, monitored, and investigated**. This creates a connection between offensive security testing and defensive security operations.

WebXploit — Web Application Attack Simulation & Defense Toolkit was developed as a controlled cybersecurity laboratory to demonstrate this complete attack-and-defense lifecycle. The project combines an intentionally vulnerable web application with a Web Application Firewall and a security monitoring pipeline. The environment uses **Kali Linux** for controlled security testing and **Ubuntu Server** for hosting the protected application and supporting security services.

The protected application is based on **OWASP Juice Shop**, while **Nginx, ModSecurity, and the OWASP Core Rule Set (CRS)** provide the Web Application Firewall layer. Controlled attack scenarios are performed against the application to observe how security rules identify potentially malicious HTTP requests and how the WAF responds to them.

The project also incorporates security telemetry and monitoring components using **Filebeat, Elasticsearch, and Kibana**. This provides a foundation for collecting security events, analyzing request information, and presenting security data for investigation.

The overall WebXploit workflow is:

Attack Simulation → Detection → Blocking → Logging → Telemetry → Monitoring → Investigation → Incident Response

Through this laboratory, the project provides practical exposure to **web application security, WAF technology, attack detection, security monitoring, detection engineering, and incident response** within an isolated and controlled environment.

3. Problem Statement

Modern web applications are exposed to a wide range of security threats because they communicate with users and external systems through network and HTTP interfaces. Attacks such as **Cross-Site Scripting (XSS)**, **SQL Injection**, **Command Injection**, **Path Traversal**, and **reconnaissance** can target vulnerable application behavior and potentially affect the confidentiality, integrity, or availability of application resources.

For cybersecurity students, understanding these attacks only from a theoretical perspective does not provide sufficient practical knowledge of how attacks appear in real network and application traffic or how defensive technologies respond to them. There is a need for a **controlled and isolated environment** where security attacks can be safely simulated and their corresponding defensive mechanisms can be observed.

Another important challenge is that detecting an attack is only one part of the security process. A complete defensive workflow should be able to identify suspicious requests, apply security rules, block malicious traffic when appropriate, generate security logs, collect telemetry, monitor events, and support subsequent investigation and incident response.

Therefore, the problem addressed by WebXploit is the lack of an integrated laboratory environment that demonstrates the complete relationship between **web application attack simulation and defensive security operations**.

WebXploit addresses this problem by integrating an intentionally vulnerable web application, Web Application Firewall, attack detection rules, security telemetry collection, centralized event storage, visualization, and investigation capabilities into a controlled laboratory environment.

The project therefore focuses on demonstrating the following security lifecycle:

Attack → Detection → Defense → Logging → Monitoring → Investigation → Response

The environment is designed strictly for authorized cybersecurity training and testing, using intentionally vulnerable applications and isolated laboratory infrastructure.

4. Objective

The primary objective of **WebXploit — Web Application Attack Simulation & Defense Toolkit** is to develop a controlled cybersecurity laboratory that demonstrates the complete lifecycle of web application attack simulation, detection, defense, security monitoring, and incident investigation.

The specific objectives of the project are:

1. **To build an isolated cybersecurity laboratory** using virtual machines and containerized services for safe and controlled security testing.
2. **To deploy an intentionally vulnerable web application** that can be used to perform authorized web application security testing.
3. **To implement a Web Application Firewall (WAF)** using Nginx and ModSecurity to inspect incoming HTTP requests.
4. **To integrate the OWASP Core Rule Set (CRS)** for detecting common web application attack patterns.
5. **To simulate controlled security attack scenarios**, including:
 - Cross-Site Scripting (XSS)
 - SQL Injection
 - Network reconnaissance
 - Command Injection
 - Path Traversal
6. **To analyze WAF detection behavior** and understand how security rules identify potentially malicious requests.

7. **To demonstrate defensive blocking mechanisms** by observing how malicious requests are rejected by the WAF.
8. **To collect and process security telemetry** generated by the web application security infrastructure.
9. **To centralize security events** using Elasticsearch for searching and analysis.
10. **To visualize and investigate security events** using Kibana.
11. **To develop practical detection engineering skills** by examining security rules, anomaly scores, request information, and WAF-generated events.
12. **To demonstrate an incident response workflow** for analyzing detected attacks and documenting appropriate defensive actions.
13. **To evaluate the effectiveness of the implemented security controls** through controlled attack scenarios and documented evidence.
14. **To create comprehensive technical documentation** containing the architecture, implementation process, attack scenarios, detection results, troubleshooting steps, and security findings.
15. **To publish the project as a reproducible cybersecurity portfolio project** with supporting documentation and evidence on GitHub.

5. Scope

The scope of **WebXploit — Web Application Attack Simulation & Defense Toolkit** covers the development of a controlled and isolated environment for simulating web application security attacks and demonstrating their detection, blocking, logging, monitoring, and investigation.

The project focuses on the interaction between **offensive security testing and defensive security controls** rather than performing unrestricted penetration testing against real-world systems.

5.1 In-Scope Activities

The following activities are included within the scope of the project:

A. Laboratory Infrastructure

- Creation of an isolated virtualized cybersecurity laboratory.
- Configuration of Kali Linux as the security testing environment.
- Configuration of Ubuntu Server as the primary server environment.
- Configuration of VMware virtual networks for communication between laboratory systems.
- Deployment of services using Docker containers.

B. Web Application Security

The project uses **OWASP Juice Shop** as the intentionally vulnerable web application for controlled security testing.

The following attack scenarios are included:

- Cross-Site Scripting (XSS)
- SQL Injection
- Network reconnaissance
- Command Injection
- Path Traversal

C. Web Application Firewall

The project includes implementation and evaluation of:

- Nginx

- ModSecurity
- OWASP Core Rule Set (CRS)
- WAF request inspection
- Rule-based attack detection
- Anomaly scoring
- Malicious request blocking

D. Security Telemetry

The project covers the collection and analysis of WAF-generated security events using:

- Filebeat
- Elasticsearch
- Kibana

The telemetry workflow is intended to provide visibility into security events such as:

- Source IP address
- HTTP request method
- Requested URI
- HTTP response status
- WAF action
- ModSecurity transaction information
- Triggered security rules

E. Security Monitoring and Investigation

The project includes:

- Security event monitoring
- Event searching and analysis
- Attack identification
- WAF evidence analysis
- Investigation of suspicious requests
- Documentation of security incidents
- Basic incident response workflow

F. Documentation and Evidence

The project includes documentation of:

- System architecture
 - Laboratory configuration
 - Implementation procedures
 - Attack scenarios
 - Detection and defense mechanisms
 - Security monitoring
 - Investigation results
 - Troubleshooting activities
 - Screenshots and technical evidence
-

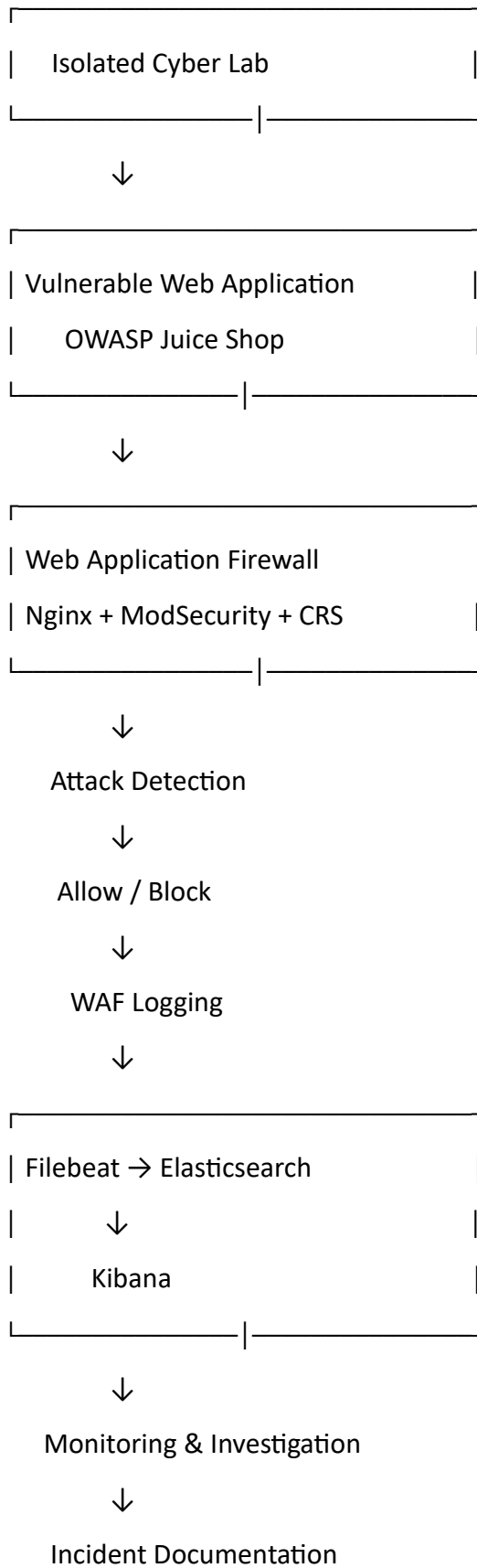
5.2 Out-of-Scope Activities

The following activities are outside the scope of this project:

- Testing unauthorized systems or applications.
- Testing production infrastructure.
- Attacking third-party organizations or websites.
- Denial-of-Service (DoS/DDoS) testing.
- Destructive exploitation.
- Theft or misuse of real user credentials.
- Testing systems without explicit authorization.
- Production deployment of the laboratory security architecture.
- Comprehensive enterprise-scale performance benchmarking.
- Guaranteeing protection against every possible web application attack.

All security testing performed as part of WebXploit is restricted to the **authorized and isolated laboratory environment**.

5.3 Scope Summary : The project can be summarized by the following workflow:



6. Technologies Used

The WebXploit project uses a combination of **virtualization, operating systems, containerization, web application security, WAF, telemetry, SIEM, and security testing technologies**. Each technology has a specific role in building and operating the laboratory.

6.1 Technology Overview

No.	Technology / Tools	Role in WebXploit
1	VMware	Provides the virtualized laboratory infrastructure
2	Kali Linux	Security testing and attack simulation platform
3	Ubuntu Server	Server platform for the WebXploit environment
4	Docker	Containerization and isolation of project services
5	Docker Compose	Deployment and management of multiple containers
6	OWASP Juice Shop	Intentionally vulnerable web application used as the testing target
7	Nginx	Web server/reverse proxy and WAF frontend
8	ModSecurity	Web Application Firewall engine
9	OWASP Core Rule Set (CRS)	Provides rules for detecting common web attacks
10	Filebeat	Security telemetry and log collection
11	Elasticsearch	Centralized storage and search of security events
12	Kibana	Visualization, monitoring, and investigation of security events
13	cURL	HTTP request generation and WAF validation
14	Nmap	Network and service reconnaissance
15	Linux Shell	System administration, automation, configuration, and troubleshooting

6.2 VMware

VMware is used as the virtualization platform for creating the isolated laboratory environment.

The project uses separate virtual machines for the security testing and server components.

The laboratory uses two VMware virtual networks:

- **VMnet8** — NAT network providing Internet connectivity.
- **VMnet1** — Host-only network providing isolated communication between the laboratory virtual machines.

This separation allows the project to maintain controlled communication between the attack and server environments.

6.3 Kali Linux

Kali Linux is used as the security testing and attack simulation platform.

It provides security assessment tools required for controlled testing of the WebXploit environment.

In the laboratory, Kali acts as the attack/test host and communicates with the Ubuntu server through the isolated network.

Configured host-only address:

Kali Linux

192.168.56.128/24

Kali is used for activities such as:

- HTTP request testing
 - XSS testing
 - SQL Injection testing
 - Command Injection testing
 - Path Traversal testing
 - Network reconnaissance
 - Security validation
-

6.4 Ubuntu Server

Ubuntu Server is used as the primary server environment for WebXploit.

It provides the operating system environment in which the Docker-based security infrastructure is deployed.

The host-only address used by the laboratory is:

Ubuntu Server

192.168.56.20/24

Ubuntu provides the underlying environment for:

- Docker
- Docker Compose
- WebXploit containers
- WAF
- Web application
- Elasticsearch
- Kibana
- Filebeat

6.5 Docker

Docker is used to containerize the WebXploit components.

Containerization provides isolation between services and simplifies deployment and management.

The project uses Docker containers for components including:

WAF

Juice Shop

Elasticsearch

Kibana

Filebeat

Docker also allows the project architecture to be reproduced without installing every application directly on the Ubuntu host.

6.6 Docker Compose

Docker Compose is used to define and manage multiple WebXploit services.

It allows the project components to be configured using declarative YAML configuration and managed as a single application stack.

The Compose-based architecture simplifies:

- Container creation
- Network configuration
- Environment configuration
- Service dependencies
- Volume configuration
- Service startup and shutdown

6.7 OWASP Juice Shop

OWASP Juice Shop is used as the intentionally vulnerable web application and attack target.

It provides a controlled environment for demonstrating common web application security vulnerabilities.

Within WebXploit, Juice Shop is positioned behind the Web Application Firewall so that HTTP requests pass through the security inspection layer before reaching the application.

6.8 Nginx

Nginx provides the HTTP frontend for the protected application.

Within WebXploit, Nginx works together with ModSecurity to inspect incoming HTTP traffic before forwarding permitted requests to the backend application.

The basic traffic flow is:

Kali



Nginx



ModSecurity



OWASP CRS



Juice Shop

6.9 ModSecurity

ModSecurity is the Web Application Firewall engine used by WebXploit.

It inspects HTTP requests and responses and evaluates them against configured security rules.

Its role in the project includes:

- HTTP request inspection
- Security rule evaluation
- Attack detection
- Anomaly scoring
- Request blocking
- Security event generation

The project uses ModSecurity with Nginx.

6.10 OWASP Core Rule Set

The **OWASP Core Rule Set (CRS)** provides the detection rules used by ModSecurity.

The project uses CRS to identify patterns associated with attacks such as:

- Cross-Site Scripting
- SQL Injection
- Command Injection
- Path Traversal
- Other malicious HTTP request patterns

For example, the SQL Injection testing performed during the project triggered CRS rule **942100**, followed by the blocking/anomaly rule **949110**.

6.11 Filebeat

Filebeat is used as the telemetry collection component.

Its role is to collect security-related log information generated by the WAF and forward the events to Elasticsearch.

The intended telemetry pipeline is:

WAF Logs



Filebeat



Elasticsearch

The project also uses Filebeat processors to normalize relevant fields for security analysis.

6.12 Elasticsearch

Elasticsearch is used as the centralized event storage and search platform.

Security telemetry collected from the WAF is indexed into Elasticsearch so that events can be searched and analyzed.

Example information that can be stored includes:

- Source IP
- HTTP method
- Request URI
- HTTP response status
- WAF action
- ModSecurity transaction information
- Event identifier
- Detection information

6.13 Kibana

Kibana provides the visualization and investigation interface for the security telemetry.

It can be used to:

- Search security events
- Filter events
- Examine individual WAF transactions
- Analyze attack activity
- Create visualizations
- Build security dashboards

The intended monitoring architecture is:

Filebeat



Elasticsearch



Kibana

6.14 cURL

cURL is used to generate controlled HTTP requests against the WebXploit WAF.

It is particularly useful for testing specific payloads and observing the resulting HTTP response.

For example, controlled SQL Injection testing can generate a request that results in:

HTTP 403 Forbidden

when the WAF blocks the request.

6.15 Nmap

Nmap is used for controlled network and service reconnaissance.

It allows the project to identify exposed ports and services on the laboratory target.

Example:

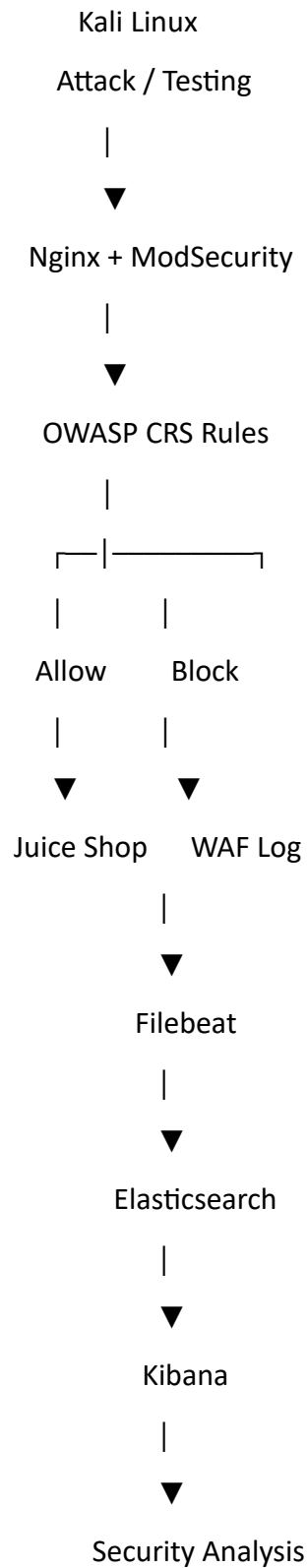
```
nmap -sV -p 80,443,8080 192.168.56.20
```

The Nmap scenario is treated primarily as a **reconnaissance and visibility test**, rather than claiming that the entire Nmap scan was blocked by the WAF.

6.16 Technology Integration

The technologies work together as a single security pipeline:

WEBXPLOIT TECHNOLOGY STACK



This technology stack provides the foundation for the subsequent chapters covering **system architecture, implementation, attack scenarios, detection and defense, security monitoring, and incident response**.

7. System Architecture

This chapter describes the architecture of **WebXploit — Web Application Attack Simulation & Defense Toolkit**, including the laboratory network, application security layer, containerized services, telemetry pipeline, and security monitoring components.

The architecture is designed around a controlled attack-and-defense workflow in which security testing traffic originates from Kali Linux, passes through the Web Application Firewall, and reaches the intentionally vulnerable web application. Security events generated by the WAF are then collected and forwarded to the monitoring stack for analysis.

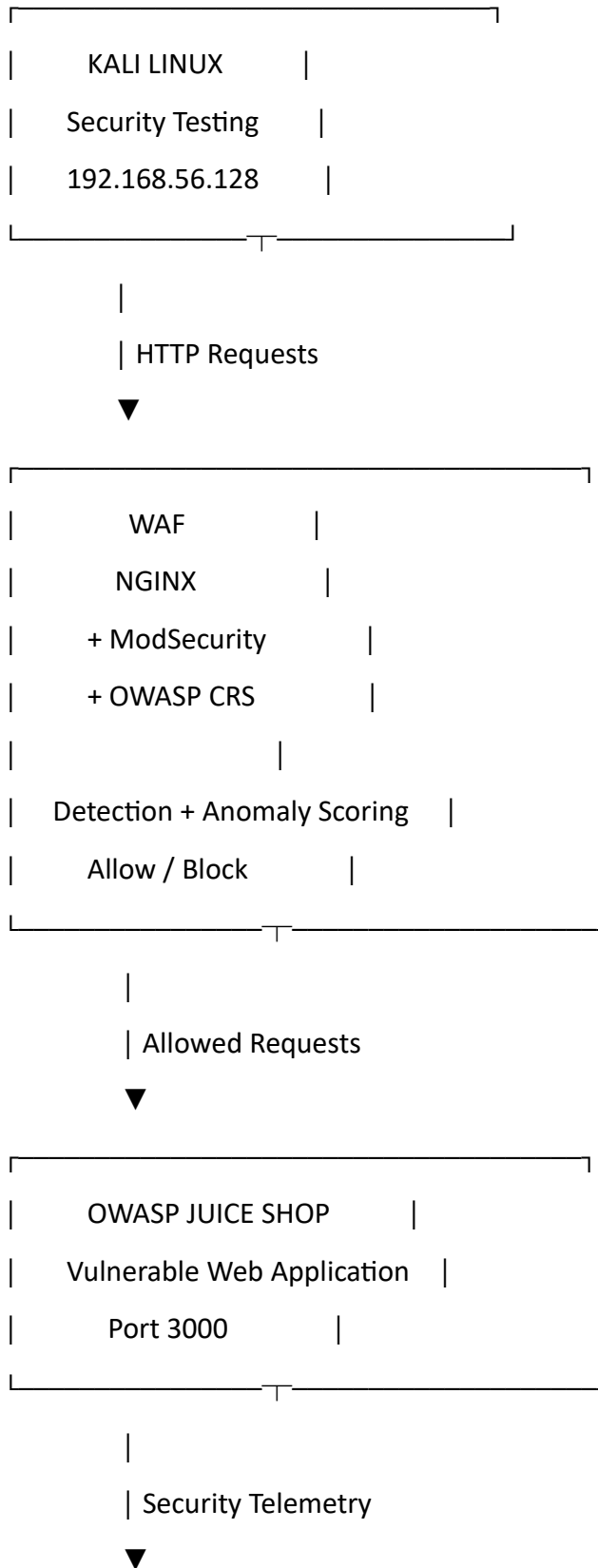
7.1 Overall System Architecture

The overall WebXploit architecture consists of the following major components:

1. **Kali Linux** — attack and security testing host
2. **Ubuntu Server** — server and container host
3. **Nginx** — HTTP frontend
4. **ModSecurity** — Web Application Firewall
5. **OWASP CRS** — WAF detection rules
6. **OWASP Juice Shop** — intentionally vulnerable target application
7. **Filebeat** — telemetry collection
8. **Elasticsearch** — centralized event storage
9. **Kibana** — security monitoring and visualization

The high-level architecture is:

WEBXPLOIT SYSTEM ARCHITECTURE



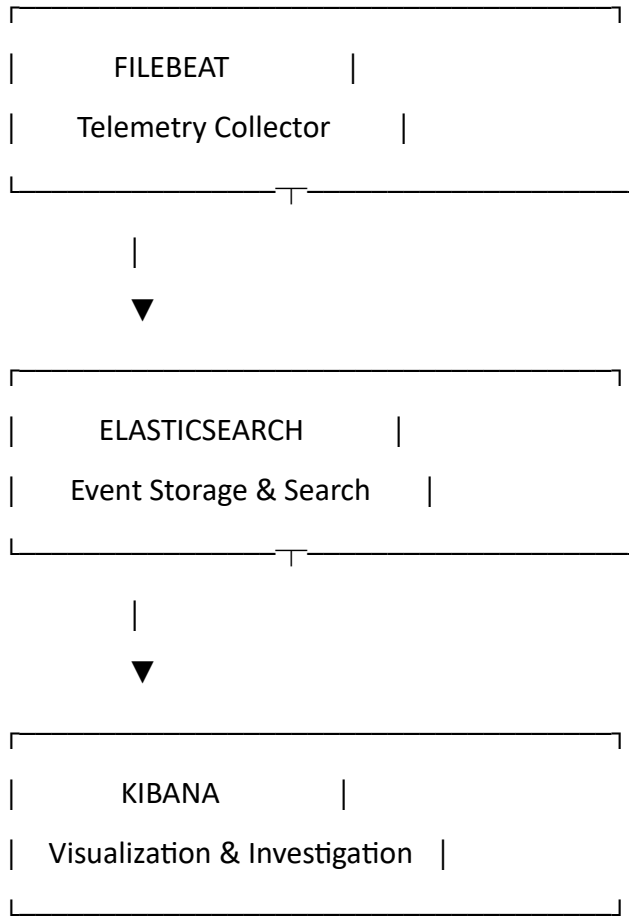


Figure 1. Overall WebXploit system architecture

Screenshot/diagram: Insert the final architecture diagram here. We should create a clean professional version later rather than using the ASCII diagram in the final PDF.

7.2 Network Architecture

The laboratory uses VMware to provide the virtualized infrastructure.

Two virtual networks are used:

VMnet8 — NAT Network

Network:

192.168.253.0/24

This network provides Internet connectivity to the virtual machines.

VMnet1 — Host-Only Network

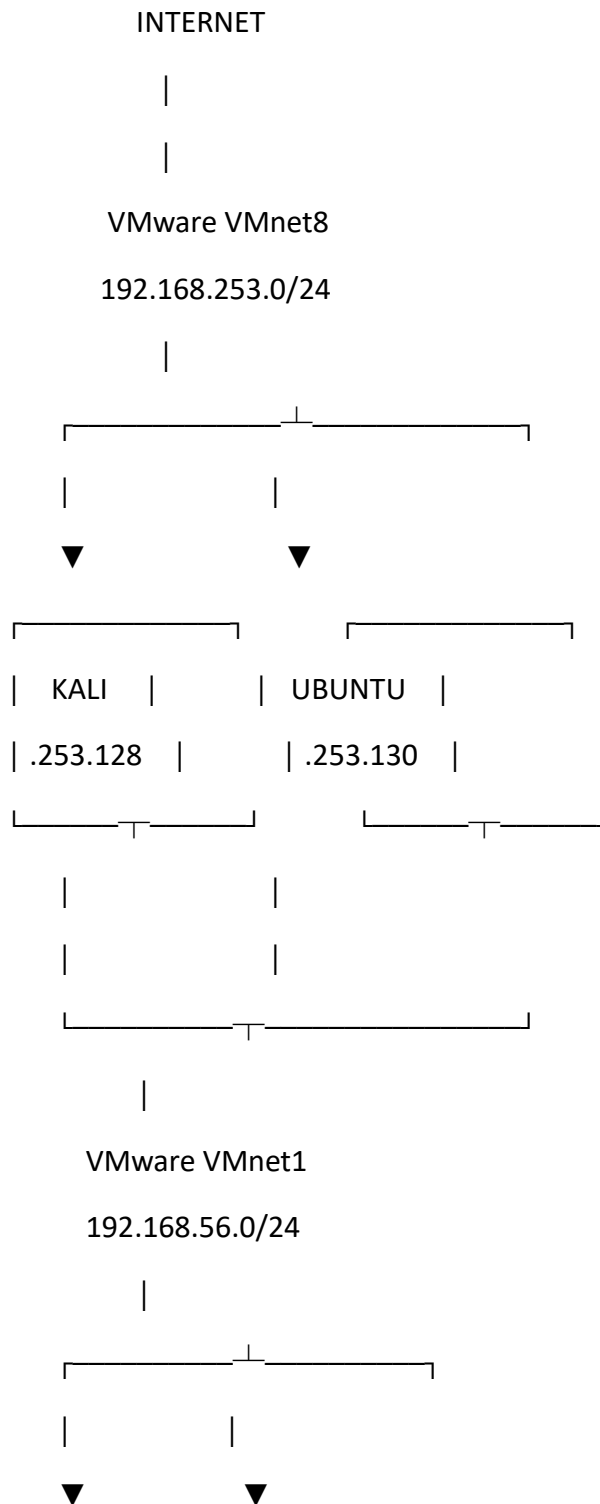
Network:

192.168.56.0/24

This network provides controlled communication between the security testing and server environments.

The primary laboratory communication takes place through the host-only network.

7.3 Network Topology



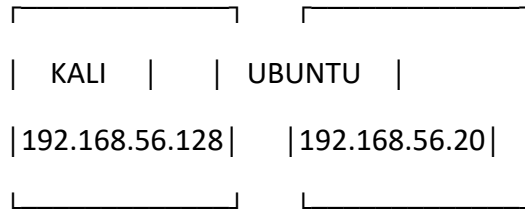


Figure 2. WebXploit laboratory network topology

The host-only network is particularly important because it provides the controlled communication path used for security testing.

7.4 Attack Traffic Flow

The attack traffic follows this path:

Kali Linux

|

| Malicious HTTP Request

▼

Nginx

|

▼

ModSecurity

|

▼

OWASP CRS

|

|— Detection

|

|— Anomaly Score

|

|— Allow

|

└── Block

|



HTTP 403

When a request is considered malicious and the configured anomaly threshold is exceeded, the WAF can reject the request.

For example, during the SQL Injection scenario, the request triggered CRS rule **942100**, followed by the blocking/anomaly rule **949110**, resulting in an HTTP 403 response.

7.5 Application Traffic Flow

Requests that pass the WAF are forwarded to the backend application:

Client

|



Nginx

|



ModSecurity

|



OWASP CRS

|

| Allowed

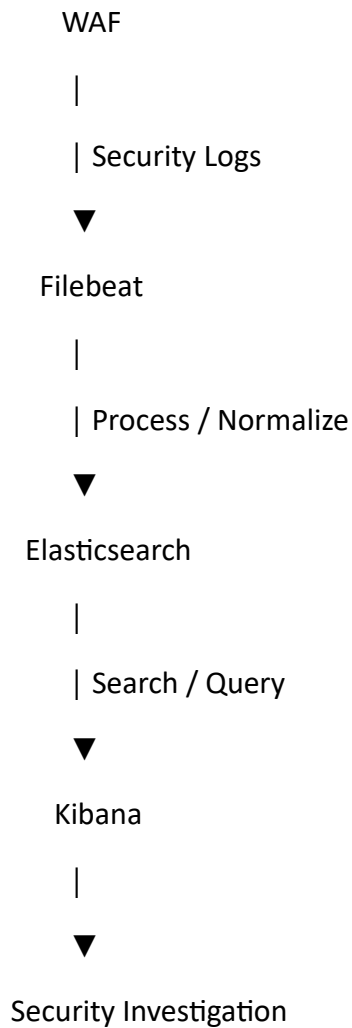


Juice Shop

The WAF therefore acts as a security control positioned between the testing client and the vulnerable application.

7.6 Security Telemetry Flow

The security monitoring architecture is designed to collect WAF-generated telemetry and make it available for centralized analysis.

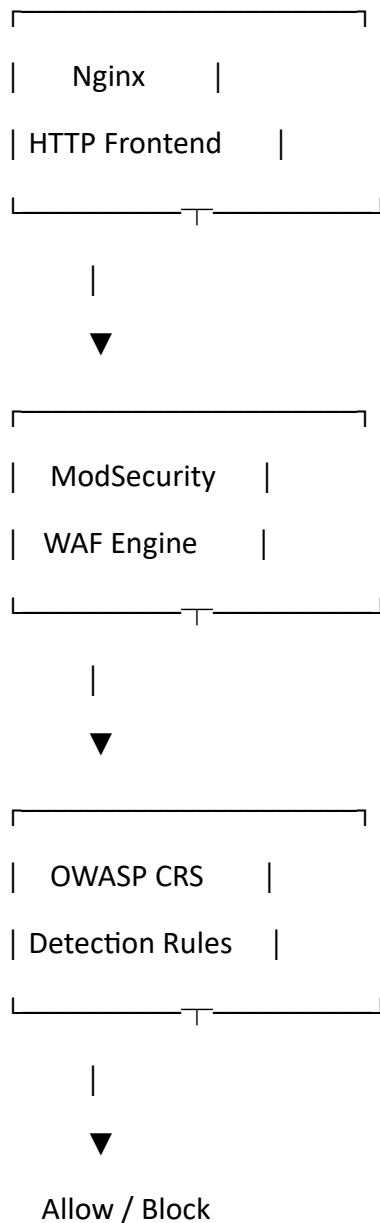


The telemetry contains information useful for security investigation, including:

- Source IP address
 - HTTP request method
 - Requested URI
 - HTTP response status
 - WAF action
 - ModSecurity transaction information
 - Event identifier
 - Detection information
-

7.7 WAF Architecture

The Web Application Firewall consists of three primary components:

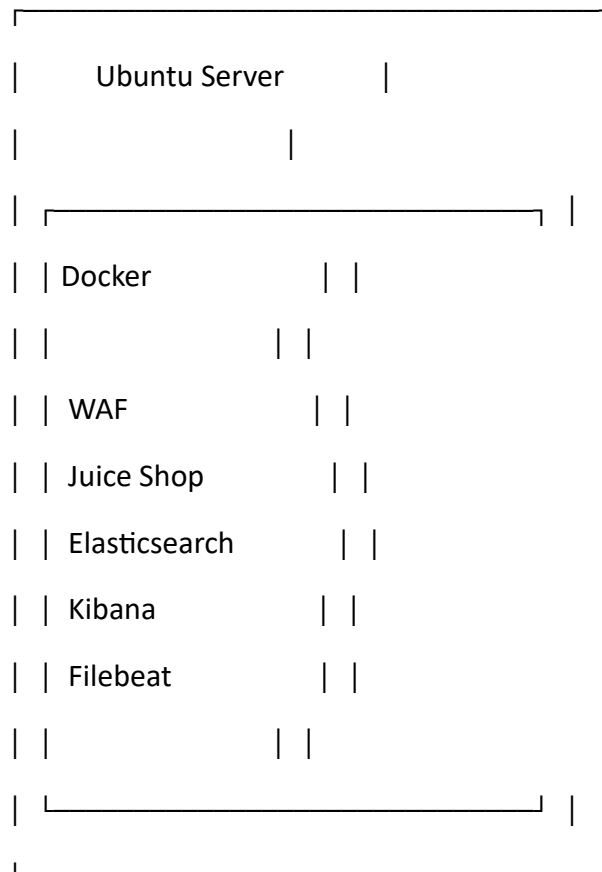


ModSecurity performs the request inspection while OWASP CRS provides the rules used to identify common malicious request patterns.

7.8 Container Architecture

The project uses Docker to isolate the major application and security components.

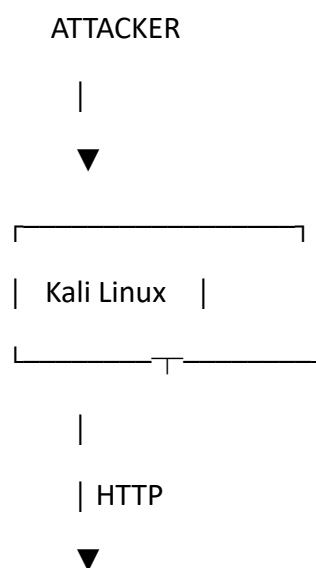
The primary services include:

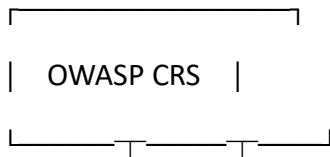
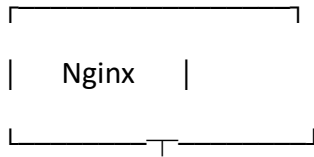


Docker networks provide communication between the appropriate services while keeping the components logically separated.

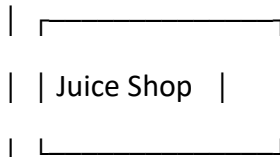
7.9 End-to-End Security Workflow

The complete WebXploit architecture can be represented as:





Block Allow



WAF Logging



Filebeat



Elasticsearch

|



Kibana

|



Investigation

|



Incident Response

This architecture demonstrates the core principle of WebXploit:

The same security event generated during attack simulation can progress from detection and blocking to centralized monitoring and investigation.

7.10 Architecture Evidence

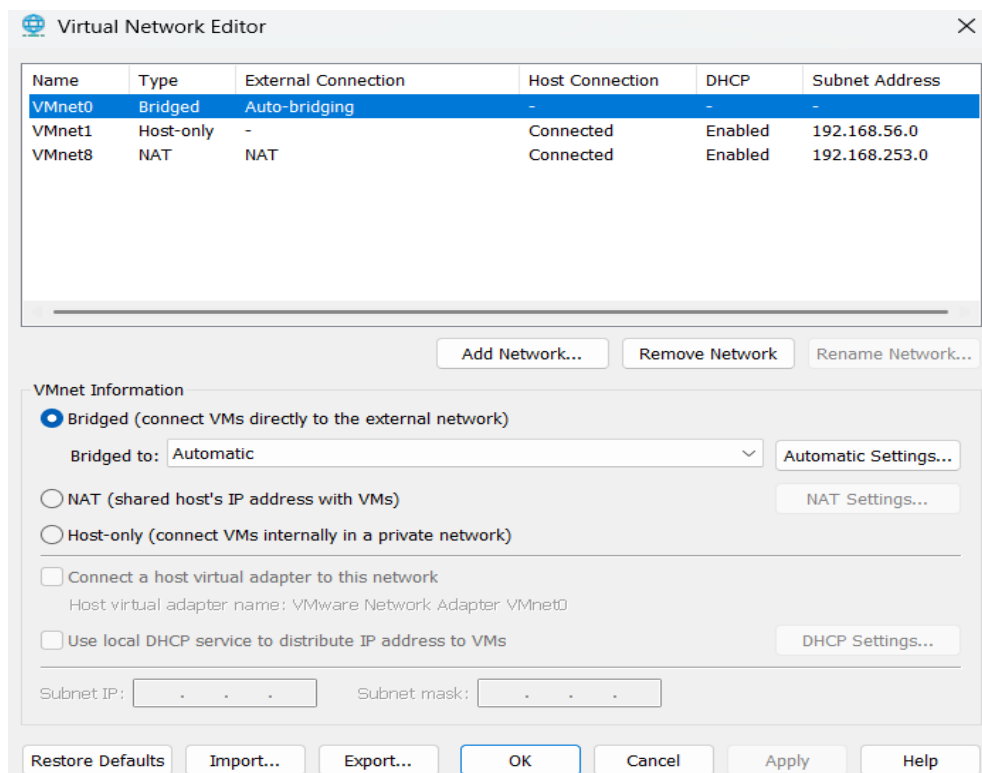


Figure: VMware Virtual Network Configuration

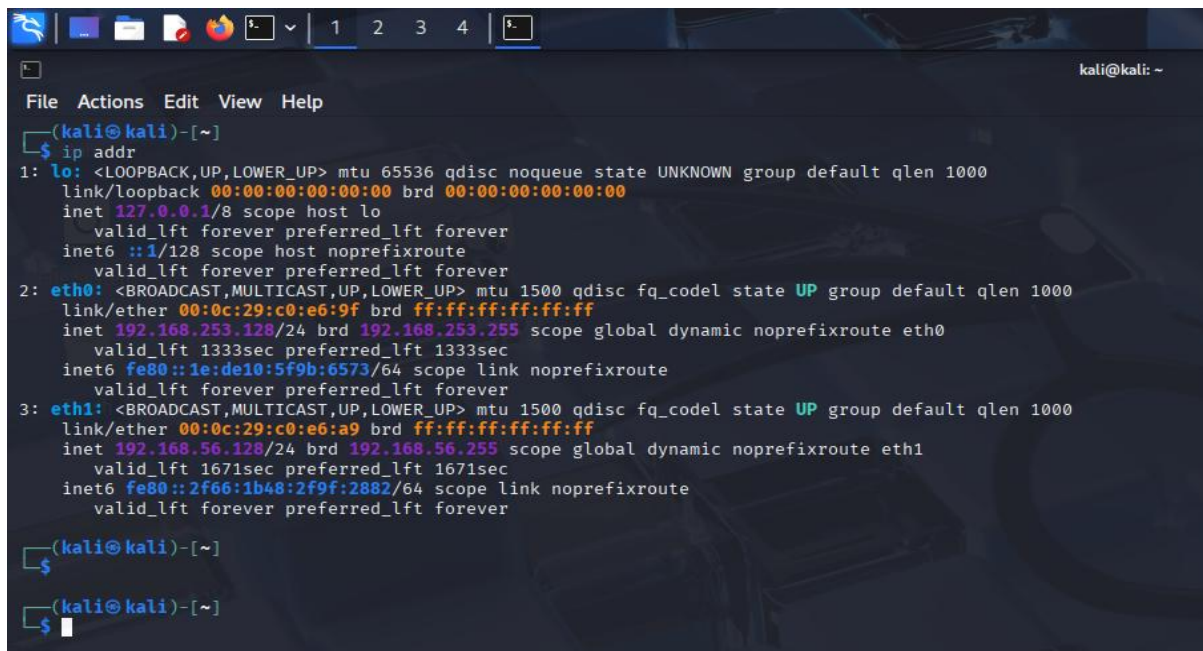
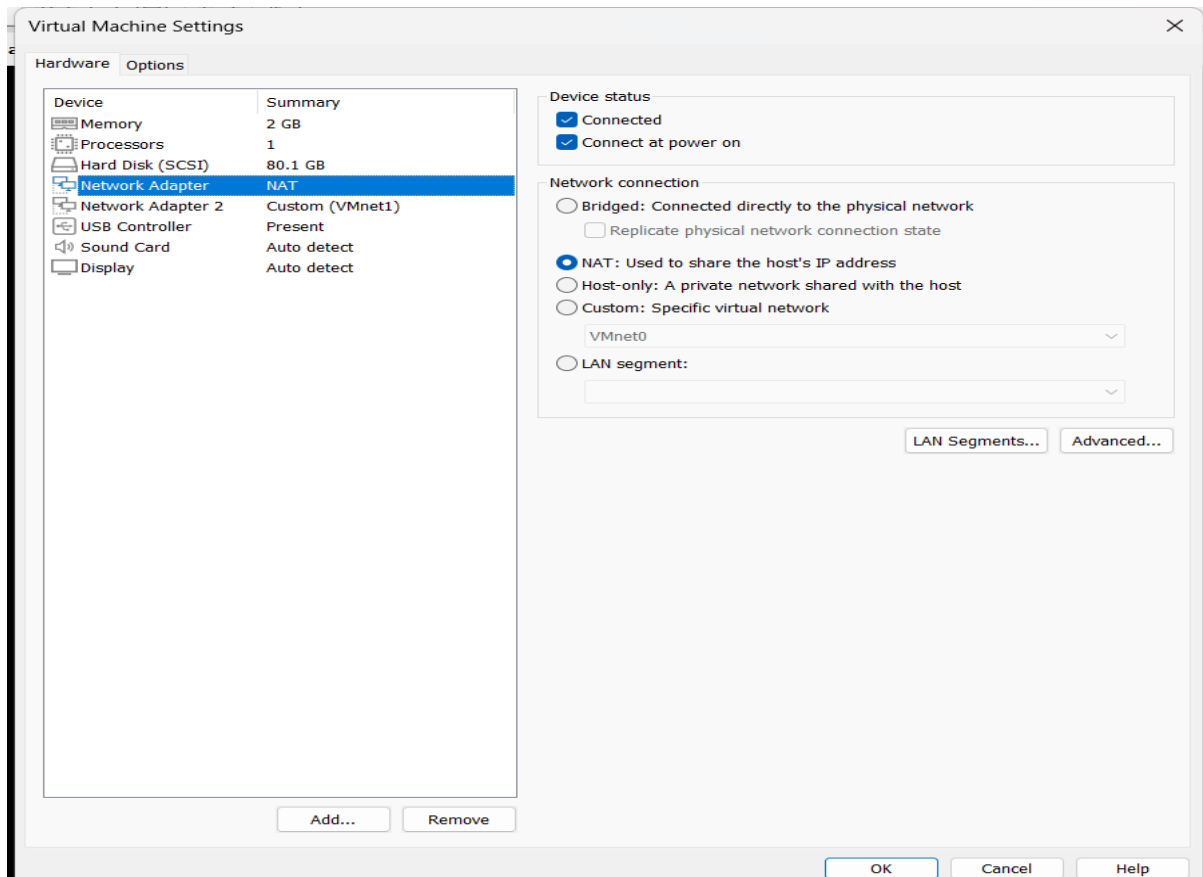


Figure : Kali Linux Network Configuration

```
(kali㉿kali)-[~]
$ ip route
default via 192.168.253.2 dev eth0 proto dhcp src 192.168.253.128 metric 102
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.128 metric 101
192.168.253.0/24 dev eth0 proto kernel scope link src 192.168.253.128 metric 102
```

Figure : Kali Linux Routing Configuration

```
(kali㉿kali)-[~]
$ ping -c 4 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
64 bytes from 192.168.56.20: icmp_seq=1 ttl=64 time=1494 ms
64 bytes from 192.168.56.20: icmp_seq=2 ttl=64 time=463 ms
64 bytes from 192.168.56.20: icmp_seq=3 ttl=64 time=283 ms
64 bytes from 192.168.56.20: icmp_seq=4 ttl=64 time=0.574 ms

— 192.168.56.20 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3279ms
rtt min/avg/max/mdev = 0.574/560.174/1494.380/563.958 ms, pipe 2

(kali㉿kali)-[~]
$
```

Figure: - Kali → Ubuntu connectivity

```
admin1@admin:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:80:42:23 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.253.130/24 metric 100 brd 192.168.253.255 scope global dynamic ens33
        valid_lft 1739sec preferred_lft 1739sec
    inet6 fe80::20c:29ff:fe80:4223/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:80:42:2d brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.56.20/24 brd 192.168.56.255 scope global ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe80:422d/64 scope link
        valid_lft forever preferred_lft forever
4: br-10686fb5255b: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 5a:13:cb:82:84:5a brd ff:ff:ff:ff:ff:ff
    inet 172.20.0.1/16 brd 172.20.255.255 scope global br-10686fb5255b
        valid_lft forever preferred_lft forever
    inet6 fe80::5813:cbff:fe82:845a/64 scope link
        valid_lft forever preferred_lft forever
5: br-48e55251c2ee: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 42:6a:2d:35:3d:d1 brd ff:ff:ff:ff:ff:ff
    inet 172.18.0.1/16 brd 172.18.255.255 scope global br-48e55251c2ee
        valid_lft forever preferred_lft forever
    inet6 fe80::406a:2dff:fe35:3dd1/64 scope link
        valid_lft forever preferred_lft forever
6: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c2:30:09:bc:3e:ba brd ff:ff:ff:ff:ff:ff
```

Figure : Ubuntu Server Network Configuration

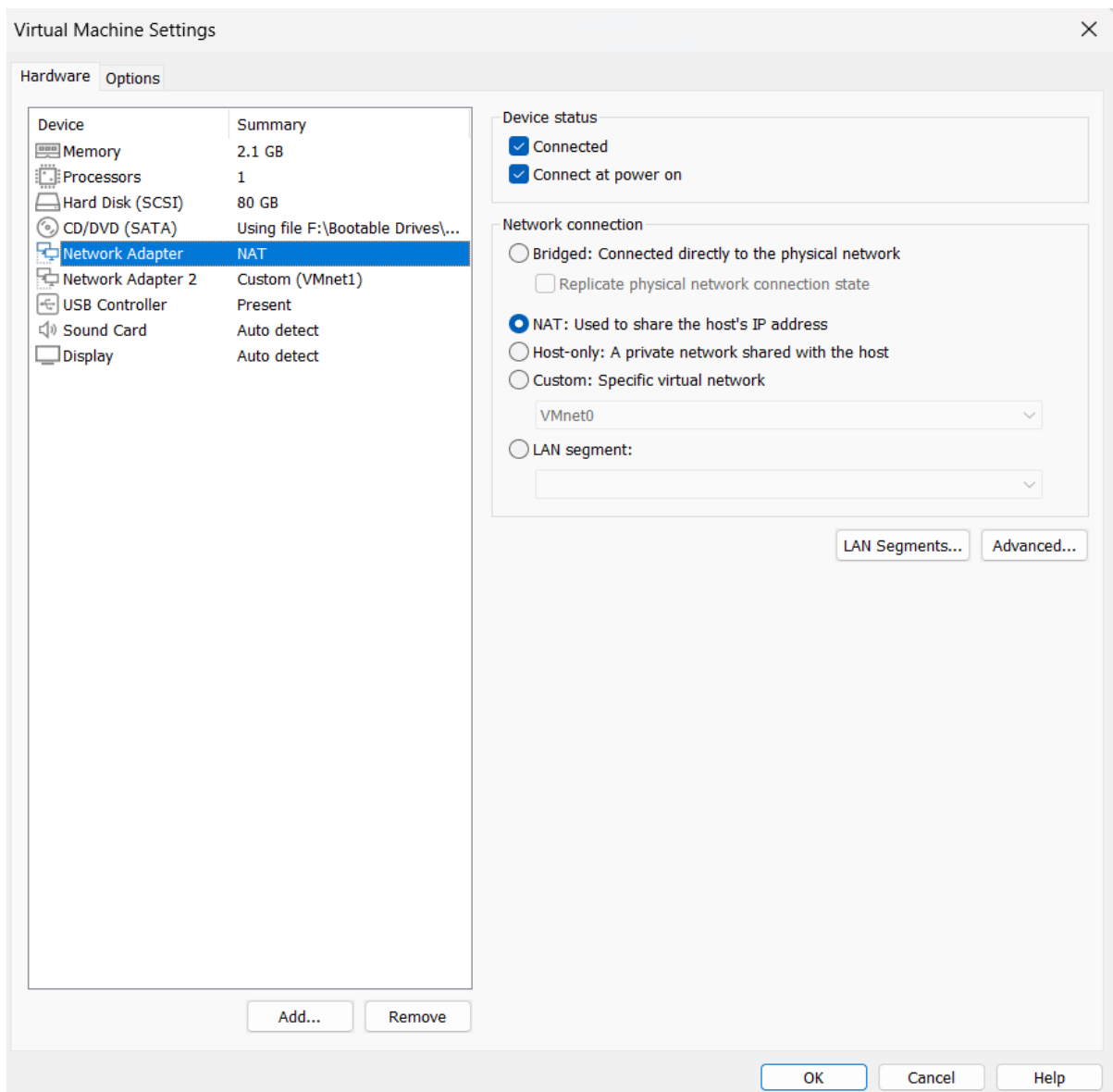


Figure : Ubuntu Network Configuration

```
admin1@admin:~$ ip route
default via 192.168.253.2 dev ens33 proto dhcp src 192.168.253.130 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-48e55251c2ee proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-dee6459efff5 proto kernel scope link src 172.19.0.1
172.20.0.0/16 dev br-10686fb5255b proto kernel scope link src 172.20.0.1
192.168.56.0/24 dev ens37 proto kernel scope link src 192.168.56.20
192.168.253.0/24 dev ens33 proto kernel scope link src 192.168.253.130 metric 100
192.168.253.2 dev ens33 proto dhcp scope link src 192.168.253.130 metric 100
admin1@admin:~$
```

Figure :- Ubuntu Routing Configuration

```

admin1@admin:~$ ip route
default via 192.168.253.2 dev ens33 proto dhcp src 192.168.253.130 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
172.18.0.0/16 dev br-48e55251c2ee proto kernel scope link src 172.18.0.1
172.19.0.0/16 dev br-dee6459efff5 proto kernel scope link src 172.19.0.1
172.20.0.0/16 dev br-10686fb5255b proto kernel scope link src 172.20.0.1
192.168.56.0/24 dev ens37 proto kernel scope link src 192.168.56.20
192.168.253.0/24 dev ens33 proto kernel scope link src 192.168.253.130 metric 100
192.168.253.2 dev ens33 proto dhcp scope link src 192.168.253.130 metric 100
admin1@admin:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=24.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=36.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=49.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=25.4 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3246ms
rtt min/avg/max/mdev = 24.872/34.179/49.802/10.175 ms
admin1@admin:~$ ping -c 4 google.com
PING google.com (142.250.134.138) 56(84) bytes of data.
64 bytes from fx-in-f138.1e100.net (142.250.134.138): icmp_seq=1 ttl=128 time=37.3 ms
64 bytes from fx-in-f138.1e100.net (142.250.134.138): icmp_seq=2 ttl=128 time=41.2 ms
64 bytes from fx-in-f138.1e100.net (142.250.134.138): icmp_seq=3 ttl=128 time=36.9 ms
64 bytes from fx-in-f138.1e100.net (142.250.134.138): icmp_seq=4 ttl=128 time=41.0 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 36.949/39.091/41.188/1.992 ms
admin1@admin:~$ |

```

Figure :- Ubuntu Internet Connectivity

```

admin1@admin:~$ docker --version
Docker version 29.8.1, build 4a63305
admin1@admin:~$ docker compose version
Docker Compose version v5.5.1
admin1@admin:~$

```

Figure:- Docker version

```

admin1@admin:~$ cd ~/WebXploit
docker ps
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED        STATUS        PORTS
abf19ddd311d   docker.elastic.co/kibana/kibana:9.2.0   webxploit-kibana   "/bin/tini -- /usr/L..." 20 hours ago   Up About a minute   0.0.0.0:
5601->5601/tcp, [::]:5601->5601/tcp
0ae3cbe875dc   docker.elastic.co/elasticsearch/elasticsearch:9.2.0   webxploit-elasticsearch   "/bin/tini -- /usr/L..." 20 hours ago   Up About a minute (health: starting)   0.0.0.0:
9200->9200/tcp, [::]:9200->9200/tcp, 9300/tcp
0c9d0e29a69a   owasp/modsecurity-crs:nginx             webxploit-waf       "/docker-entrypoint..." 24 hours ago   Up About a minute (healthy)           0.0.0.0:
80->8080/tcp, [::]:80->8080/tcp
6af3212eac66   bkimminich/juice-shop:latest            webxploit-juice-shop   "/nodejs/bin/node /j..." 24 hours ago   Up About a minute   3000/tcp
admin1@admin:~/WebXploit$ |

```

Figure :- Docker services

```

admin1@admin:~/WebXploit$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
e2c8bacfbddb   bridge   bridge    local
25fc89b800c1   host     host      local
79ee9a8d1c36   none     null      local
10686fb5255b   telemetry_webxploit_telemetry   bridge    local
dee6459efff5   webxploit_backend               bridge    local
48e55251c2ee   webxploit_frontend              bridge    local
admin1@admin:~/WebXploit$

```

Figure : Docker networks

Project Implementation

9. Project Implementation

The WebXploit environment was implemented as a layered cybersecurity laboratory consisting of a virtualized infrastructure, containerized applications, a Web Application Firewall, and a security monitoring stack. The implementation was performed in multiple stages, beginning with the laboratory network and ending with security telemetry and monitoring.

9.1 Network Configuration

Two VMware networks were configured:

- **VMnet8 NAT:** 192.168.253.0/24
- **VMnet1 Host-only:** 192.168.56.0/24

Kali Linux was configured as the security testing host, while Ubuntu Server was configured as the WebXploit server.

The host-only network provides the controlled communication path between Kali and Ubuntu, while the NAT network provides Internet connectivity for system administration and software installation.

9.2 Docker Environment

Docker was installed on Ubuntu Server to provide an isolated and reproducible deployment environment.

The WebXploit services were organized into separate containers and Docker networks. Docker Compose was used to define and manage the multi-service environment.

The primary components include:

- WAF
- OWASP Juice Shop
- Elasticsearch
- Kibana
- Filebeat

9.3 Vulnerable Web Application

OWASP Juice Shop was deployed as the intentionally vulnerable target application.

The application is positioned behind the WAF so that HTTP requests are inspected before reaching the backend application.

The resulting request flow is:

Kali



Nginx + ModSecurity



OWASP CRS



Juice Shop

9.4 Web Application Firewall

Nginx was deployed with ModSecurity and the OWASP Core Rule Set.

The WAF performs HTTP request inspection and evaluates requests against security rules. When a request matches malicious patterns and exceeds the configured anomaly threshold, the request is blocked.

During testing, the WAF successfully generated detection and blocking events for scenarios including XSS, SQL Injection, Command Injection, and Path Traversal.

For example, the SQL Injection scenario triggered CRS rule **942100**, followed by the anomaly/blocking rule **949110**, resulting in an HTTP 403 response.

9.5 Security Telemetry

WAF security events are collected for centralized monitoring using the telemetry stack.

The intended data flow is:

WAF



Filebeat



Elasticsearch



Kibana

Relevant event information includes source IP address, HTTP method, requested URI, response status, WAF action, and ModSecurity transaction information.

9.6 Implementation Summary

The completed architecture therefore combines:

Virtualization



Network Isolation



Docker



Vulnerable Application



WAF + CRS



Security Logging



Telemetry



SIEM / Visualization

10. Attack Scenarios

10.1 Overview

WebXploit was designed to simulate common web application attack techniques against the intentionally vulnerable OWASP Juice Shop application in an isolated laboratory environment. The objective was to evaluate whether the Web Application Firewall (WAF), based on Nginx, ModSecurity and OWASP Core Rule Set (CRS), could detect and block malicious requests.

ID	Attack Scenario	Primary Detection	Result
001	Cross-Site Scripting (XSS)	CRS 941xxx	Blocked – HTTP 403
002	SQL Injection	CRS 942100	Blocked – HTTP 403
003	Network Reconnaissance	Nginx/WAF telemetry	Observed
004	Command Injection	CRS 932xxx	Blocked – HTTP 403
005	Path Traversal	CRS 930xxx	Blocked – HTTP 403

10.2 Cross-Site Scripting (XSS)

A controlled XSS payload was submitted through the WebXploit WAF.

The WAF detected multiple XSS-related patterns, including script-tag and JavaScript injection indicators. The request exceeded the configured anomaly threshold and was rejected.

Result: HTTP 403 Forbidden

Relevant CRS rules:

- 941100 – XSS via libinjection
- 941110 – XSS script-tag vector
- 941160 – HTML/JavaScript injection detection
- 949110 – Inbound Anomaly Score Evaluation

Security outcome: The malicious request was blocked before reaching the backend application.

10.3 SQL Injection

A controlled SQL injection payload was submitted using the following request pattern:

```
/?id=1' OR '1'='1
```

The WAF identified the SQL injection pattern using the CRS SQL injection detection rules.

Relevant rule:

942100 – SQL Injection Attack Detected via libinjection

The request generated an anomaly score above the configured threshold and was blocked.

Result: HTTP 403 Forbidden

Security outcome: The SQL injection request was prevented from reaching the Juice Shop application.

10.4 Network Reconnaissance

Nmap was used from the Kali Linux system to perform controlled reconnaissance against the WebXploit server.

Example:

```
nmap -sV -p 80,443,8080 192.168.56.20
```

The scan identified the HTTP service exposed on port 80.

The purpose of this scenario was to demonstrate how reconnaissance activity appears in the network and web-service telemetry.

Result: Reconnaissance activity was observed and logged. The test was not treated as a successful application-layer attack.

10.5 Command Injection

A controlled command-injection payload was submitted through the web application.

The request contained a shell command separator and command execution pattern.

The WAF detected multiple command-injection signatures from the OWASP CRS 932xxx rule family.

Relevant rules included:

- 932235
- 932260
- 932340
- 932380
- 949110

Result: HTTP 403 Forbidden

Security outcome: The malicious request was blocked by ModSecurity before reaching the backend application.

10.6 Path Traversal

Controlled path traversal requests were submitted using both direct and URL-encoded traversal sequences.

Example:

`/?file=../../../../etc/passwd`

The WAF detected Unix path traversal patterns.

Relevant CRS rules:

- 930100

- 930110
- 930120
- 932160
- 949110

Result: HTTP 403 Forbidden

Security outcome: The WAF prevented the traversal request from being processed by the backend application.

Detection and Defense

WebXploit uses a layered defensive mechanism to detect and prevent malicious web requests. The primary defensive layer consists of **Nginx, ModSecurity and the OWASP Core Rule Set (CRS)**.

11.1 Detection Mechanism

Incoming HTTP requests first pass through the Nginx reverse proxy. ModSecurity inspects the request and applies OWASP CRS rules to identify suspicious patterns associated with common web attacks.

The detection process is:

Client Request → Nginx → ModSecurity → OWASP CRS → Anomaly Scoring → Allow/Block

When a request matches one or more security rules, ModSecurity assigns an anomaly score. If the score exceeds the configured inbound threshold, the request is rejected.

In the WebXploit environment, the inbound anomaly threshold was configured as **5**.

11.2 Attack Detection

The implemented CRS rules detected patterns associated with:

- Cross-Site Scripting
- SQL Injection
- Command Injection
- Path Traversal
- Other suspicious HTTP requests

For example, SQL injection testing triggered **CRS rule 942100**, while command injection triggered rules from the **932xxx** family.

11.3 Request Blocking

When the anomaly score exceeded the configured threshold, ModSecurity returned:

HTTP/1.1 403 Forbidden

The request was therefore stopped at the WAF layer instead of being forwarded to the Juice Shop backend.

11.4 Logging and Evidence

The WAF generated security and access information containing details such as:

- Request method
- Requested URI
- Detection rule
- Anomaly score
- HTTP response status
- Client information
- Unique transaction identifier

These logs provide the foundation for subsequent security monitoring and investigation.

11.5 Defense Workflow

Incoming Request



Nginx



ModSecurity



OWASP CRS



Rule Matching



Anomaly Score



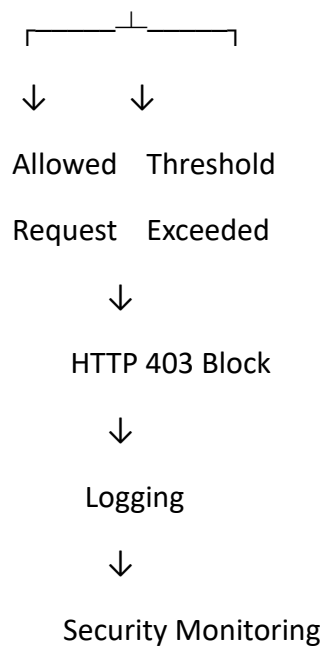


Figure 11.1 — WebXploit Detection and Defense Workflow

11.6 Defensive Outcome

The testing demonstrated that the WebXploit WAF could identify multiple common attack patterns and block malicious requests before they reached the vulnerable application. The generated logs also provide security telemetry for subsequent monitoring and investigation.

Chapter 12 — Security Monitoring

WebXploit incorporates centralized security monitoring to provide visibility into WAF and web application activity.

12.1 Telemetry Pipeline

The intended monitoring pipeline is:

WAF → Filebeat → Elasticsearch → Kibana

- **WAF:** Generates security and access logs.
- **Filebeat:** Collects and forwards log data.
- **Elasticsearch:** Stores and indexes telemetry.
- **Kibana:** Provides visualization and investigation capabilities.

12.2 Monitoring Objectives

The monitoring layer is designed to support:

- Detection of malicious requests

- Analysis of attack patterns
- Review of WAF decisions
- Event investigation
- Identification of repeated attack activity
- Visualization of security events

12.3 Security Investigation

Analysts can use the collected telemetry to examine individual events and correlate information such as:

Timestamp

Client IP

HTTP Method

Request URI

Attack Pattern

CRS Rule ID

Anomaly Score

HTTP Status

This provides a foundation for investigating security incidents generated during the attack simulations.

12.4 Monitoring Architecture

Juice Shop

↑

Nginx + ModSecurity

|

↓

WAF Logs

|

↓

Filebeat

|

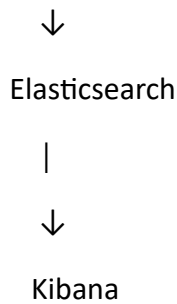


Figure 12.1 — WebXploit Security Monitoring Pipeline

12.5 Monitoring Result

The monitoring architecture demonstrates how application-layer security events can be collected and centralized for analysis. During implementation, telemetry configuration and connectivity limitations were identified and documented for future improvement.

13 — Incident Response Workflow

WebXploit follows a simplified incident response process to demonstrate how a security analyst can handle detected web attacks.

13.1 Incident Response Lifecycle

The workflow consists of the following stages:

Detection → Validation → Investigation → Containment → Analysis → Recovery → Documentation

13.2 Detection

A suspicious request is identified by ModSecurity and OWASP CRS. The corresponding WAF event provides information such as the request URI, detected rule and anomaly score.

13.3 Validation and Investigation

The security event is reviewed to determine:

- Source of the request
- Target URI
- Attack pattern
- Triggered CRS rule
- Anomaly score
- HTTP response

- Related events

The analyst determines whether the event represents a genuine attack attempt or expected traffic.

13.4 Containment

For malicious requests detected during testing, the WAF blocks the request by returning **HTTP 403**. This prevents the request from reaching the vulnerable Juice Shop application.

13.5 Analysis and Documentation

Security events and WAF logs are analyzed to understand the attack technique and effectiveness of the defensive controls. Important findings are recorded as part of the incident documentation.

13.6 Recovery

After the controlled attack simulation, the laboratory environment is returned to its normal state. Since the project operates in an isolated test environment, no production systems are affected.

13.7 Incident Response Workflow

Attack Request



Detection by WAF



Validate Security Event



Investigate Logs



Contain / Block



Analyze Attack



Document Findings



Improve Detection

14 — Results and Analysis

The WebXploit implementation was evaluated by executing controlled web attack scenarios against the Juice Shop application through the configured WAF.

Scenario	Detection	Response	Result
XSS	CRS 941xxx	HTTP 403	Blocked
SQL Injection	CRS 942100	HTTP 403	Blocked
SQL Injection	Service/WAF telemetry	Observed	Detected/Observed
Command Injection	CRS 932xxx	HTTP 403	Blocked
Path Traversal	CRS 930xxx	HTTP 403	Blocked

14.2 Security Analysis

The testing demonstrated the following:

1. **XSS detection:** The WAF identified JavaScript and script-injection patterns.
2. **SQL injection detection:** CRS rule 942100 detected the SQL injection payload.
3. **Reconnaissance visibility:** Nmap activity provided observable network/service information.
4. **Command injection detection:** Multiple CRS 932xxx rules identified command-execution patterns.
5. **Path traversal detection:** CRS 930xxx rules identified traversal attempts.
6. **Blocking:** Malicious requests that exceeded the anomaly threshold were rejected with HTTP 403.
7. **Logging:** WAF activity generated security telemetry that can be used for investigation.

14.3 Overall Result

The project successfully demonstrated an end-to-end security testing workflow:

Attack Simulation



WAF Detection



Rule Matching



Anomaly Scoring



Request Blocking



Security Logging



Monitoring & Investigation

The results show that the implemented defensive controls were effective against the controlled attack patterns tested in the laboratory environment.

14.4 Key Findings

Area	Findings
WAF	Successfully inspected HTTP requests
OWASP CRS	Detected multiple attack categories
Blocking	Malicious requests returned HTTP 403
Logging	Security events were generated
Monitoring	Centralized monitoring architecture was implemented
Incident Response	Controlled response workflow was demonstrated

15 — Challenges and Troubleshooting

During implementation, several technical challenges were encountered.

15.1 Virtual Disk Capacity

The initial Ubuntu virtual disk had insufficient capacity for the security monitoring stack. The virtual disk was expanded and the Linux partition/LVM filesystem was extended.

15.2 Docker Networking

Multiple Docker networks were required to separate application, WAF and telemetry components while maintaining controlled communication between services.

15.3 WAF Configuration

ModSecurity and OWASP CRS required configuration and validation before attack simulations could be performed successfully.

15.4 Telemetry Pipeline

Filebeat Docker autodiscovery experienced Docker socket communication timeouts during testing. The issue was documented as a telemetry implementation limitation.

15.5 Kibana Connectivity

Kibana was operational locally, while access through the Ubuntu host-only address experienced connectivity issues. This was treated as a network-path limitation rather than blocking completion of the core WAF testing.

15.6 Lessons Learned

The troubleshooting process provided practical experience with:

- VMware networking
- Linux storage management
- Docker networking
- Reverse proxies
- WAF configuration
- Security logging
- SIEM/telemetry architecture
- Network troubleshooting

Chapter 16 — Limitations

The current WebXploit implementation has several limitations:

- The environment is designed for laboratory use and not production deployment.
- The attack scenarios are controlled and limited in scope.
- Telemetry ingestion requires further refinement.
- Detection rules are primarily based on OWASP CRS.
- Advanced correlation and automated response capabilities are not implemented.
- The project does not currently provide complete enterprise-scale monitoring.
- Testing was performed against intentionally vulnerable applications in an isolated environment.

These limitations provide opportunities for future development.

Chapter 17 — Future Enhancements

Future versions of WebXploit can include:

1. Complete Filebeat-to-Elasticsearch log ingestion.
2. Advanced Kibana security dashboards.
3. Custom ModSecurity detection rules.
4. Additional web attack scenarios.
5. Automated attack simulation scripts.
6. Alerting and notification mechanisms.
7. Integration with a SIEM platform such as Splunk.
8. Threat intelligence integration.
9. Automated incident-response actions.
10. MITRE ATT&CK technique mapping.
11. CI/CD security testing.
12. Additional vulnerable applications and attack targets.

The project can therefore evolve from a laboratory demonstration into a more comprehensive web-security detection and response platform.

Chapter 18 — Conclusion

WebXploit was developed as an isolated web application security laboratory for demonstrating the relationship between offensive security testing and defensive monitoring.

The project integrated **Kali Linux, Docker, OWASP Juice Shop, Nginx, ModSecurity, OWASP CRS, Filebeat, Elasticsearch and Kibana** into a controlled security environment.

Multiple attack scenarios, including **XSS, SQL injection, reconnaissance, command injection and path traversal**, were simulated. The WAF successfully detected and blocked the tested malicious requests where applicable, returning HTTP 403 responses and generating security logs.

The project also demonstrated the importance of centralized telemetry, security monitoring and incident response in a web application security environment.

Overall, WebXploit provided practical experience in **web security, WAF deployment, attack simulation, detection engineering, logging, monitoring and incident response**.

Chapter 19 — References

1. OWASP Foundation — OWASP Top 10 Web Application Security Risks.
2. OWASP Foundation — OWASP Core Rule Set (CRS).
3. OWASP Foundation — OWASP Juice Shop.
4. OWASP ModSecurity — ModSecurity Web Application Firewall.
5. Nginx Documentation — Nginx Web Server and Reverse Proxy.
6. Elastic — Elasticsearch Documentation.
7. Elastic — Kibana Documentation.
8. Elastic — Filebeat Documentation.
9. Docker Documentation — Docker Engine and Docker Compose.
10. VMware Documentation — VMware Workstation Networking.